Quaternion Posture Correction for a Quadruped Robot

A derivation of quaternion angle error, angular-velocity PD feedback, stationary-contact foot kinematics, and body-to-leg coordinate transformations

Purpose: provide a defensible theoretical basis for a simple kinematic body-posture correction layer before Jacobian inversion and joint-command integration.

Project-specific theory note
Prepared 17 August 2026

Executive summary

The proposed algorithm is a kinematic posture-correction layer. The IMU quaternion and gyroscope define an attitude state. A relative quaternion is converted to a three-dimensional rotation vector. A PD law interprets this error as a desired angular acceleration, which is integrated to a corrective angular velocity. Under a no-slip stance assumption, rigid-body kinematics then determine the foot velocity relative to the body that would be compatible with the opposite body motion. Finally, that velocity is re-expressed in each leg frame using a rotation matrix. Jacobian inversion begins only after this note ends.

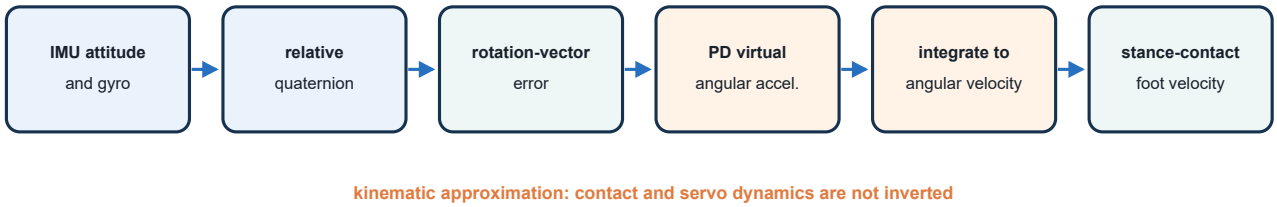


Figure 1. Logical chain of the proposed controller. The orange warning is a model boundary, not a minor implementation detail.

Central result

For stance foot i, with body-frame foot vector r_i and corrective body twist (v_B, ω_B), the required foot-coordinate velocity relative to the body is $\dot{r}_i = -v_B - \omega_B \times r_i$. For pure attitude correction, $v_B = 0$, so $\dot{r}_i = -\omega_B \times r_i$.

This result is exact at the velocity-kinematics level under the stated frame convention and no-slip point-contact assumption. It is not by itself a proof that the physical floating body will realize the commanded angular acceleration. That stronger claim requires actuator dynamics, ground-reaction forces, friction, contact switching, and the rigid-body inertia model [5, 6].

Assumptions

- All coordinate frames are right-handed unless explicitly stated otherwise.
- The measured and desired quaternions use the same convention and represent body-to-world orientation.
- Angular velocities are expressed in compatible frames; sensor mounting rotation is removed before control.
- A stance foot is a stationary, no-slip point contact during the interval in which correction is applied.
- The leg-base frame is rigidly attached to the body, so its orientation relative to the body is constant.
- The acceleration-level PD proof applies to a virtual angular-acceleration plant; the physical robot requires a contact-force realization layer for the same proof to hold.

1. Frames, notation, and the meaning of a coordinate vector

Three frames are used: the inertial/world frame {W}, the body frame {B}, and leg frame $\{L_i\}$ for leg i. A physical vector is independent of coordinates, but its numerical components depend on the frame. The notation v^B means the components of physical vector v expressed in {B}. This distinction is emphasized in Modern Robotics, which separately treats free vectors, points, and reference-frame changes [2, Ch. 3].

Symbol	Meaning
R_{WB}	Rotation mapping body coordinates into world coordinates: $v^W = R_{WB} v^B$.
R_{BL}	Rotation mapping leg-frame coordinates into body-frame coordinates.
p_B^W	Position of the body-frame origin in world coordinates.
r_{BF}^B	Vector from body origin to foot, expressed in the body frame.
$omega^B$, $alpha^B$	Body angular velocity and angular acceleration, expressed in {B}.
$[a]_x$	Skew-symmetric matrix satisfying $[a]_x b = a \times b$.
q = (eta, epsilon)	Unit quaternion with scalar part eta and vector part epsilon.

1.1 Rotation matrices

A proper three-dimensional rotation belongs to the special orthogonal group SO(3). Its columns are an orthonormal, right-handed basis. Therefore

$$R \text{ in } SO(3) \text{ iff } R^T R = I, \quad \det(R) = +1 \tag{1}$$

Orthogonality proves that $R^{-1} = R^T$. The determinant condition excludes reflections. A diagonal map diag(1,1,-1), for example, is orthogonal but has determinant -1, so it is a reflection rather than a physical rotation between right-handed frames [2, Sec. 3.2].

1.2 Points are not velocity vectors

Let p_{BL}^B be the leg-frame origin relative to the body origin. A foot point transforms by rotation plus translation:

$$p_F^B = p_{BL}^B + R_{BL} p_F^L \tag{2}$$

A velocity is a free vector. If {L} is rigidly fixed to {B}, p_{BL}^B and R_{BL} are constant when viewed from {B}. Differentiating (2) therefore removes the translation:

$$\text{p-dot}_F^B = R_{BL}\,\text{p-dot}_F^L, \quad \text{p-dot}_F^L = R_{LB}\,\text{p-dot}_F^B = R_{BL}^T\,\text{p-dot}_F^B \tag{3}$$

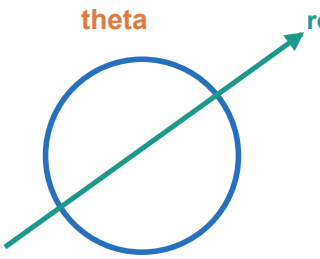
Implementation consequence

A function that adds half the body length or width is a point-position transform. It cannot be reused unchanged for velocity or acceleration. The vector transform uses only the rotation matrix.

2. Unit quaternions and physical rotation

A unit quaternion is written q = (eta, epsilon), where eta is scalar, epsilon is a three-vector, and $eta^2 + epsilon^T epsilon = 1$. For a right-handed rotation of angle theta around unit axis u, the axis-angle parameterization is [1, Secs. 2-3]

$$q = (\cos(theta/2), \quad u \sin(theta/2)), \quad ||u|| = 1 \tag{4}$$



q_e = (eta, epsilon)

eta = cos(theta/2)

epsilon = u sin(theta/2)

phi = theta u

Figure 2. A quaternion stores a half-angle in its scalar and vector parts. The rotation vector phi = theta u has the physical angle as its magnitude.

2.1 Why q and -q represent the same attitude

The rotation of vector x by a unit quaternion is represented by the quaternion product $q\ (0,x)\ q^*$. Replacing q with -q introduces two minus signs, which cancel. Therefore q and -q induce the same rotation matrix. Unit quaternions are a double cover of SO(3) [1, 4]. A controller that ignores this fact can command an unnecessary long rotation, known as unwinding [4].

$$(-q)(0,x)(-q)^* = q(0,x)q^* \tag{5}$$

2.2 Quaternion inverse and composition

For a unit quaternion q = (eta, epsilon), the conjugate $q^* = (eta, -epsilon)$ is also the inverse. Composition is quaternion multiplication, not addition or subtraction. If q_{WB} maps body coordinates into world coordinates and q_{WD} is the desired body orientation, the relative rotation from the measured body frame to the desired frame is

$$q_e = q_{WB}^{-1} \times q_{WD} = q_{WB}^{*} \times q_{WD} \quad (\times \text{ denotes quaternion product}) \tag{6}$$

Equation (6) is the quaternion counterpart of $R_e = R_{WB}^T R_{WD}$. If q_{WB} equals q_{WD}, q_e is the identity quaternion (1,0,0,0). Quaternion subtraction has no corresponding group-error meaning and is not invariant to the sign choice q versus -q.

2.3 Shortest branch

Write $q_e = (eta_e, epsilon_e)$. Because q_e and $-q_e$ are physically identical, choose $eta_e >= 0$. Then theta lies on the principal branch [0, pi]. Computationally, negate all four coefficients whenever $eta_e < 0$. At exactly pi radians, the axis sign is intrinsically ambiguous; no continuous global three-parameter attitude coordinate exists.

3. From quaternion error to an angle-error vector

The PD controller requires a vector in R^3, while q_e lies on the unit three-sphere. The principled local coordinate is the rotation vector, also called the logarithmic coordinate or exponential coordinate:

$$\text{phi} = \text{theta}\ u = \text{Log}_{SO(3)}(R_e)^{\text{vee}} \tag{7}$$

The vee operator converts a skew-symmetric matrix back to its three-vector. The vector direction is the rotation axis; its Euclidean norm is the rotation angle in radians [1, 2].

3.1 Exact quaternion-to-angle-axis conversion

From (4), $||\text{epsilon}_e||$ = sin(theta/2) and eta_e = cos(theta/2). The two-argument arctangent preserves the correct quadrant and remains well-scaled near zero:

$$\text{theta} = 2\ \text{atan2}(||\text{epsilon}_e||, \text{eta}_e), \quad u = \text{epsilon}_e\ /\ ||\text{epsilon}_e|| \tag{8}$$

$$\text{phi} = 2\ \text{atan2}(||\text{epsilon}_e||, \text{eta}_e)\ \text{epsilon}_e/||\text{epsilon}_e|| \text{ if } ||\text{epsilon}_e|| > 0\text{; otherwise phi} = 0 \tag{9}$$

At zero rotation, every axis describes the same identity orientation, so axis() alone is not physically unique. The product angle() x axis() is nevertheless zero and is the meaningful quantity. In Eigen, `AngleAxisd(q).axis()` returns the stored normalized axis u, while `angle()` returns theta in radians [7].

3.2 Small-angle approximation

For small theta, sin(theta/2) = theta/2 + $O(\text{theta}^3)$. Therefore epsilon_e = u theta/2 + $O(\text{theta}^3)$ and

$$\text{phi} = 2\text{epsilon}_e + O(\text{theta}^3) \tag{10}$$

Thus 2 q_e.vec() is a local approximation to the exact angle-axis vector. The exact angle-axis expression is preferable for a wide correction range; the approximation is computationally simple but should be paired with a small-error operating envelope.

> Convention check
>
> The sign and multiplication order depend on whether the stored quaternion maps body-to-world or world-to-body and on whether angular velocity is expressed in body or world coordinates. A one-axis test is mandatory: tilt the robot a small positive roll from a level desired attitude and verify that the computed error and commanded corrective motion have the intended signs.

4. Angular velocity error and quaternion kinematics

For body-to-world quaternion q_{WB} and body-resolved angular velocity omega^B, one common Hamilton-convention kinematic equation is [1]

$$\text{q-dot}_{WB} = (1/2)\, q_{WB} \times (0, \text{omega}^B) \tag{11}$$

This equation is convention-sensitive; reversing the mapping direction or quaternion multiplication convention changes its form. The practical invariant requirement is that all compared angular velocities be expressed in the same frame.

4.1 Correct frame transport for a moving desired attitude

Let $R_e = R_{WB}{}^T R_{WD}$. If desired angular velocity $\text{omega}_d{}^D$ is expressed in the desired frame {D}, transport it into the measured body frame before subtraction:

$$e_{omega} = R_e \text{omega}_d{}^D - \text{omega}^B \tag{12}$$

For a fixed desired level attitude, $\text{omega}_d = 0$ and (12) reduces to $e_{omega} = -\text{omega}^B$. Simple component-wise subtraction is valid only when both vectors already share a frame. Raw BNO055 gyro data are sensor-frame data until a fixed sensor-to-body mounting rotation is applied.

5. PD control as a virtual angular-acceleration law

Let e_R = phi from (7)-(9), pointing from the measured attitude toward the desired attitude, and use e_{omega} from (12). A diagonal-gain acceleration-level PD law is

$$\text{alpha}_c = K_R e_R + K_{omega} e_{omega}, \quad K_R \text{ and } K_{omega} \text{ positive definite} \tag{13}$$

K_R has units s^{-2} and K_{omega} has units s^{-1} when alpha_c is in rad/s^2. The command should be saturated because neither contact forces nor joint velocities are unlimited.

5.1 Local stability proof for the virtual plant

Assume: (i) the desired attitude is fixed; (ii) attitude error is small enough that $\text{e-dot}_R = e_{omega}$ + higher-order terms; and (iii) the realized body angular acceleration equals alpha_c. Because $e_{omega} = -\text{omega}$, $\text{e-dot}_{omega} = -\text{alpha}_c$. Ignoring higher-order terms gives

$$\text{e-dot}_R = e_{omega}, \quad \text{e-dot}_{omega} = -K_R e_R - K_{omega} e_{omega} \tag{14}$$

Choose the positive-definite candidate

$$V = (1/2) e_{omega}{}^T e_{omega} + (1/2) e_R{}^T K_R e_R \tag{15}$$

Differentiating along (14) yields

$$\text{V-dot} = -e_{omega}{}^T K_{omega} e_{omega} <= 0 \quad \text{(cross terms cancel)} \tag{16}$$

The cross terms cancel. By LaSalle's invariance principle, the only invariant set near the desired equilibrium with V-dot = 0 is $e_{omega} = 0$ and $e_R = 0$; hence the local virtual error dynamics are asymptotically stable. Lee gives a nonlinear geometric treatment directly on SO(3), including large-error tracking and rigid-body dynamics [3].

What this proof does not prove

It does not prove stability of the complete quadruped under foot-position commands. Assumption (iii) is precisely the missing dynamics: contacts and actuators must make the physical body angular acceleration follow alpha_c. The proof is valid for the virtual angular-acceleration subsystem, not automatically for the hardware.

5.2 Torque-level rigid-body law

For a rigid body with body-frame inertia J and external body torque tau, Euler's equation is

$$J\dot{\omega} + \omega \times (J\omega) = \tau \tag{17}$$

A computed-torque realization of α_c would use $\tau_c = J\alpha_c + \omega \times J\omega$, plus tracking feedforward terms when the desired attitude moves. Geometric controllers formulate this rigorously and avoid Euler-angle singularities [3]. For a legged robot, however, tau is mainly produced by stance-foot ground-reaction forces, not by an abstract body torque actuator.

6. Derivation of body-to-foot velocity correction

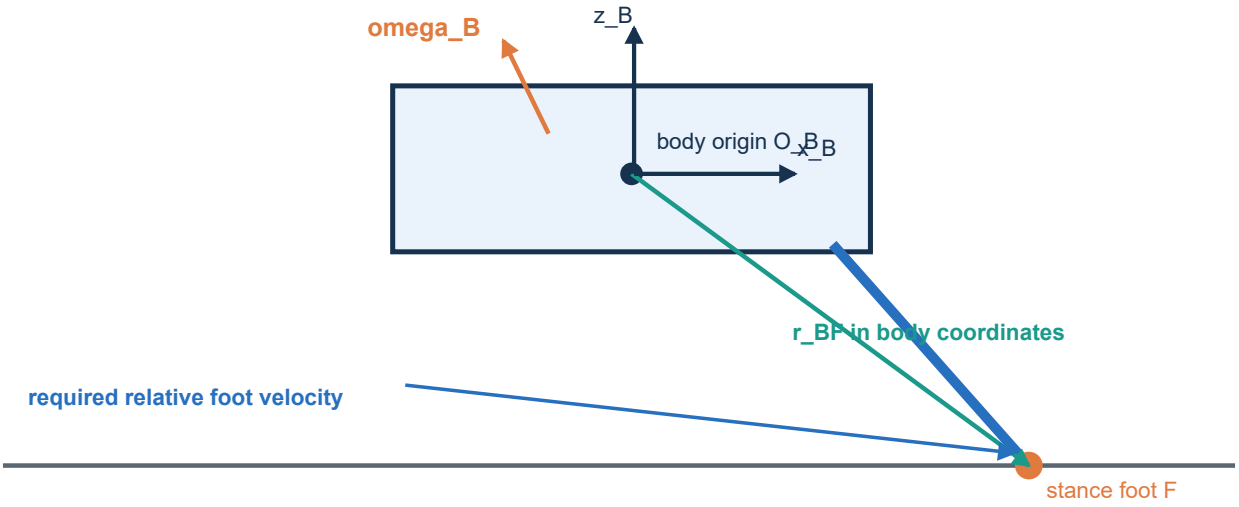


Figure 3. A stance foot constrains the floating body. The leg must change the foot coordinates relative to the body if the body moves while the world contact stays fixed.

6.1 Position identity

Let ${}^{W}p_F$ be the foot position in the world, ${}^{W}p_B$ the body-origin position, and ${}^{B}r_{BF}$ the body-to-foot vector expressed in the body frame. These describe the same physical geometry:

$${}^{W}p_F = {}^{W}p_B + R_{WB}\,{}^{B}r_{BF} \tag{18}$$

6.2 Transport theorem

The body-frame angular velocity satisfies $\dot{R}_{WB} = R_{WB}[{}^{B}\omega]_\times$. Differentiating (18) by the product rule gives

$${}^{W}v_F = {}^{W}v_B + R_{WB}({}^{B}\omega \times {}^{B}r_{BF} + {}^{B}\dot{r}_{BF}) \tag{19}$$

The first term is body-origin translation. The cross product is the velocity induced at the foot location by body rotation. The final term is the foot's velocity relative to the body, generated by changing joint angles. Equation (19) is the standard rigid-body transport theorem specialized to a moving point [2, Ch. 3].

6.3 Stationary stance constraint

Under ideal no-slip point contact, the stance foot is stationary in the world, so $v_F^W = 0$. Multiply (19) by $R_{BW} = R_{WB}^T$ and define $v_B^B = R_{BW} v_B^W$:

$$r\text{-}dot_{BF}^B = -v_B^B - omega^B \times r_{BF}^B \tag{20}$$

This is the requested body-velocity-to-foot-velocity relationship. The negative signs are not tuning choices; they follow from the stationary-contact constraint. If the body translates forward, the foot coordinates relative to the body move backward. If the body rotates, the relative foot velocity cancels omega x r. Contact-based legged-state estimators use the same stance constraint, while acknowledging errors caused by slip and contact uncertainty [5].

6.4 Pure attitude correction

If the posture layer commands no body-origin translation, set $v_B^B = 0$. Integrate the virtual angular acceleration once:

$$omega_c[k+1] = omega_c[k] + alpha_c[k]\ delta\text{-}t \tag{21}$$

Then recompute each stance-foot correction directly from current geometry:

$$r\text{-}dot_{i,c}^B = -omega_c^B \times r_i^B \tag{22}$$

Integrating one body angular-velocity state and regenerating all foot velocities from (22) preserves rigid-body consistency. Independently integrating four foot accelerations creates four drifting states that need not correspond to any single body motion.

6.5 Optional acceleration-level relation

Differentiating (20) in the rotating body frame gives the exact coordinate acceleration

$$r\text{-}ddot_{BF}^B = -v\text{-}dot_B^B - alpha^B \times r_{BF}^B - omega^B \times r\text{-}dot_{BF}^B \tag{23}$$

At low angular speed and with zero translation, the leading term is approximately -alpha x r. The omitted omega-dependent terms matter as speed increases. Equation (22), evaluated every control step, is generally the cleaner implementation for a velocity-level leg controller.

7. Converting the foot correction from body frame to leg frame

Let R_{BL_i} map components from leg frame $\{L_i\}$ to body frame $\{B\}$. For the foot point, equation (2) applies. Solving for leg-frame position gives

$$p_F^{Li} = R_{LiB}(p_F^B - p_{BLi}^B), \quad R_{LiB} = R_{BLi}^T \tag{24}$$

Because the leg-base pose is rigidly attached to the body, both $p_{BL_i}^B$ and R_{BL_i} are constant in body coordinates. Differentiate (24):

$$p\text{-}dot_F^{Li} = R_{LiB} p\text{-}dot_F^B = R_{BLi}^T p\text{-}dot_F^B \tag{25}$$

No body-length or body-width translation appears in (25). Those offsets locate the leg origin for positions, but translations do not change the coordinates of a free velocity vector. If all leg axes are aligned with body axes, R_{BL_i} may

be the identity. If axes differ, encode the difference as an explicit proper rotation matrix and verify $R^T R = I$ and $\det R = +1$.

> Current-project frame audit
>
> The current leg-to-body position mapping adds front/rear and left/right body offsets and negates only the Z component. The offsets are appropriate only for point positions. A Z-only sign flip has determinant -1, so it is not a proper rotation between two right-handed frames. Before applying (25), document whether the leg frame is left-handed or whether one more axis/sign convention is missing. Cross products and angular quantities require a right-handed convention to keep their usual signs.

8. Why stance-foot motion can influence posture - and why it is not a full dynamics controller

The kinematic constraint explains which relative foot velocity is compatible with a desired body twist. Physical posture changes, however, arise from forces. For body mass m, center-of-mass acceleration a_C, body inertia J, gravity g, and stance-foot forces f_i, Newton-Euler equations give

$$m a_C = \sum_{i \in S} f_i + mg \tag{26}$$

$$J\dot{\omega} + \omega \times (J\omega) = \sum_{i \in S} r_i \times f_i \tag{27}$$

The stance set S matters: a swing foot has no ground-reaction force and cannot directly create body torque. A force-based controller chooses feasible f_i satisfying unilateral contact and friction limits, then maps those forces to joint torques. Cheetah-family controllers explicitly distribute desired translational and angular accelerations through stance-foot forces [6], and whole-body/MPC controllers treat reaction forces and contact schedules as first-class variables [8].

8.1 What position-controlled stance feet actually do

A position or velocity servo that moves a stance leg against a non-slipping ground produces contact forces through closed-chain constraint error, actuator stiffness, mechanical compliance, and soil deformation. Those forces may rotate the body in the desired direction, but the mapping from commanded foot velocity to realized angular acceleration is not α_c = command. It depends on unmodeled dynamics and can cause foot slip, impact, or force spikes.

Consequently, the acceleration-level PD gains in (13) are kinematic shaping gains until a force or torque realization layer is introduced and validated. The local proof in Section 5 establishes the intended virtual error dynamics; it is not a certificate for the complete contact-rich hybrid robot.

8.2 Contact weighting

Let c_i in [0,1] be a contact confidence or a smooth stance weight. The correction can be blended with planned gait velocity:

$$\dot{p}_{i,cmd}^{B} = \dot{p}_{i,gait}^{B} + c_i(-v_c^{B} - \omega_c^{B} \times r_i^{B}) \tag{28}$$

Set $c_i = 0$ during swing. Smooth transitions reduce discontinuities at touchdown and liftoff. A contact detector should use more than the gait phase when hardware feedback permits, because an expected contact can be absent and an unexpected early contact can occur.

9. Mathematically consistent algorithm contract

Stage	Required mathematical contract
IMU frame	Rotate sensor-frame gyro into {B}; ensure quaternion and gyro conventions are mutually consistent.
Attitude error	$q_e = q_m^{-1} \times q_d$; choose the shortest representative; convert with the exact log/angle-axis map.
Rate error	Transport desired angular velocity into the measured body frame before subtraction; for fixed desired attitude use $-\text{omega}^B$.
PD	Compute $\text{alpha}_c = K_R e_R + K_{\text{omega}} e_{\text{omega}}$; limit alpha and integrated omega.
Contact map	For stance only, use $\text{r-dot}_i^B = -v_c^B - \text{omega}_c^B \times r_i^B$.
Frame change	Use rotation only: $\text{r-dot}_i^L = R_{LB}\,\text{r-dot}_i^B$. Do not add hip translation to a velocity.
Downstream	Then apply the leg Jacobian inverse, joint-rate limits, joint-acceleration limits, and command integration. These steps are outside this note.

10. Verification tests before hardware posture control

Test	Pass condition
Quaternion identity	Set measured equal to desired; q_e is identity, angle error is zero, and axis extraction produces no NaN.
Double cover	Replace q_d by -q_d; the physical error and command remain identical after shortest-branch handling.
One-axis sign	Apply small positive roll, pitch, and yaw errors separately; error axes and corrective foot directions have the intended signs.
Rigid contact	Choose known r and omega; v_F^W from (19) is zero when r-dot uses (20).
Frame round trip	For every leg, R_BL R_LB = I and det(R_BL) = +1; B-to-L-to-B recovers a random velocity.
Stance gating	Swing legs receive no posture-contact correction and contact weights transition smoothly.
Limits and freshness	Reject stale IMU/PDO data; bound angular acceleration, integrated angular velocity, foot correction, joint velocity, and joint acceleration.

Recommended first hardware scope

Begin with the robot restrained or supported, low roll/pitch gains, zero yaw correction, and small angular-velocity and foot-velocity limits. A stance-foot velocity command can generate large contact forces even when its numerical value looks modest.

11. Conclusions

The attitude error is obtained by group composition, not quaternion subtraction. The exact vector error is the logarithm/angle-axis vector of $q_m^{-1} q_d$; Eigen's axis() supplies its unit direction and angle() supplies its magnitude. A PD law can shape a virtual angular-acceleration subsystem with a standard local Lyapunov argument. Integrating that acceleration gives a corrective angular velocity.

The foot correction follows from differentiating the world/body position identity and imposing zero world velocity at a stance contact. This yields r-dot = -v - omega x r. Body-to-leg conversion of that velocity uses only R_{LB}; translations belong to points, not free vectors. These results rigorously justify the kinematic chain up to the leg Jacobian under explicit frame and contact assumptions.

The same derivation also exposes the limitation: a kinematic foot command is not a direct body torque. A full physical stability claim requires contact-force feasibility, inertia, actuator and servo dynamics, slip modeling, and hybrid contact transitions. The proposed controller is therefore best described as a simple stance-foot kinematic posture correction layer, with a future path toward force-distribution or whole-body control.

References

[1] J. Sola, 'Quaternion kinematics for the error-state Kalman filter,' arXiv:1711.02508, 2017. https://arxiv.org/abs/1711.02508

[2] K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press, 2017. Official book resources.

[3] T. Lee, 'Geometric Tracking Control of the Attitude Dynamics of a Rigid Body on SO(3),' Proc. American Control Conference, 2011. https://arxiv.org/abs/1010.1725

[4] C. G. Mayhew, R. G. Sanfelice, and A. R. Teel, 'On Quaternion-Based Attitude Control and the Unwinding Phenomenon,' Proc. American Control Conference, pp. 299-304, 2011, doi:10.1109/ACC.2011.5991127. Preprint.

[5] M. Bloesch, M. Hutter, M. A. Hoepflinger, S. Leutenegger, C. Gehring, C. D. Remy, and R. Siegwart, 'State Estimation for Legged Robots - Consistent Fusion of Leg Kinematics and IMU,' Robotics: Science and Systems, 2012. https://doi.org/10.3929/ethz-a-010137526

[6] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, 'MIT Cheetah 3: Design and Control of a Robust, Dynamic Quadruped Robot,' IEEE/RSJ IROS, 2018. MIT open-access manuscript.

[7] Eigen project, 'Eigen::AngleAxis Class Template Reference.' Official API documentation, accessed 17 Aug. 2026.

[8] D. Kim, J. Di Carlo, B. Katz, G. Bledt, and S. Kim, 'Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control,' arXiv:1909.06586, 2019. https://arxiv.org/abs/1909.06586

Citation note: equations are derived in this document from standard quaternion, SO(3), rigid-body transport, and Newton-Euler relations. References are supplied both for the foundational laws and for the limits introduced by legged contact dynamics.